

Étudiante : Koumba Awa KANE
Matricule : 10513413
Date : 30/08/2026
Repo GitHub : <https://github.com/AwaKane/eshop-supdeco>

RAPPORT FINAL : Examen DevSecOps

E-Shop Pro

Résumé Exécutif

E-Shop Pro est une API e-commerce en Node.js/Express/SQLite qui gère un catalogue de produits, l'authentification des utilisateurs et les commandes. Le threat modeling STRIDE a fait ressortir des menaces centrées sur le risque financier (manipulation du prix) et la compromission de données (injection SQL, secrets exposés). L'analyse manuelle du code a trouvé 13 vulnérabilités couvrant six catégories de l'OWASP Top 10, dont une injection SQL, un secret JWT codé en dur incohérent avec celui utilisé pour signer les tokens, et un défaut de contrôle d'accès (IDOR) sur les commandes et les profils.

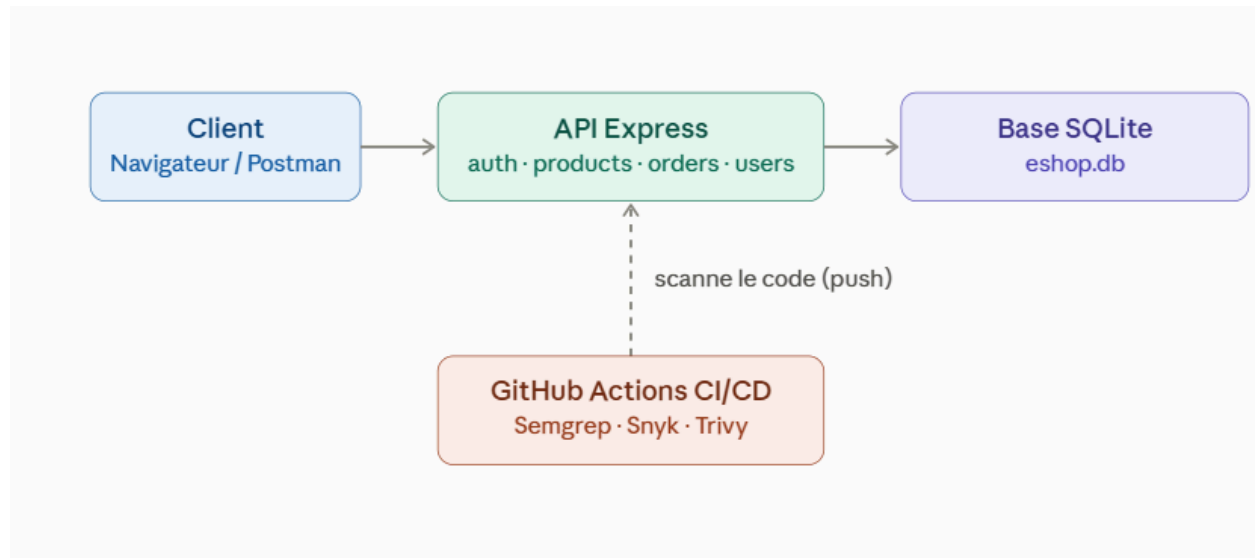
Cinq règles Semgrep ont été écrites pour ce code précis. Elles ont détecté les vulnérabilités critiques là où les règles génériques du marché n'en trouvaient qu'une seule. Une pipeline GitHub Actions à 7 jobs (GitLeaks, Semgrep, Snyk, Trivy, build Docker, résumé) bloque désormais réellement le build tant que les scans critiques ne passent pas. Quatre vulnérabilités ont été corrigées et validées, une cinquième découverte pendant les corrections. Les dépendances vulnérables ont été nettoyées : trois paquets inutilisés supprimés, les autres mis à jour. L'image Docker a aussi été sécurisée après que Trivy a révélé deux CVE critiques venant de l'image de base elle-même, pas du code. La pipeline passe entièrement au vert.

1. Présentation de l'application

E-Shop Pro est une API backend qui gère un catalogue de produits, l'authentification et les commandes, en Node.js/Express avec une base SQLite locale. L'authentification repose sur des JSON Web Tokens. Deux rôles existent, customer par défaut et admin, même si le contrôle d'accès sur ce rôle s'est révélé absent au départ (voir Partie 3).

Les données sensibles sont les emails et mots de passe des utilisateurs (stockés en clair au départ, voir plan de remédiation), les tokens JWT, et les montants des commandes.

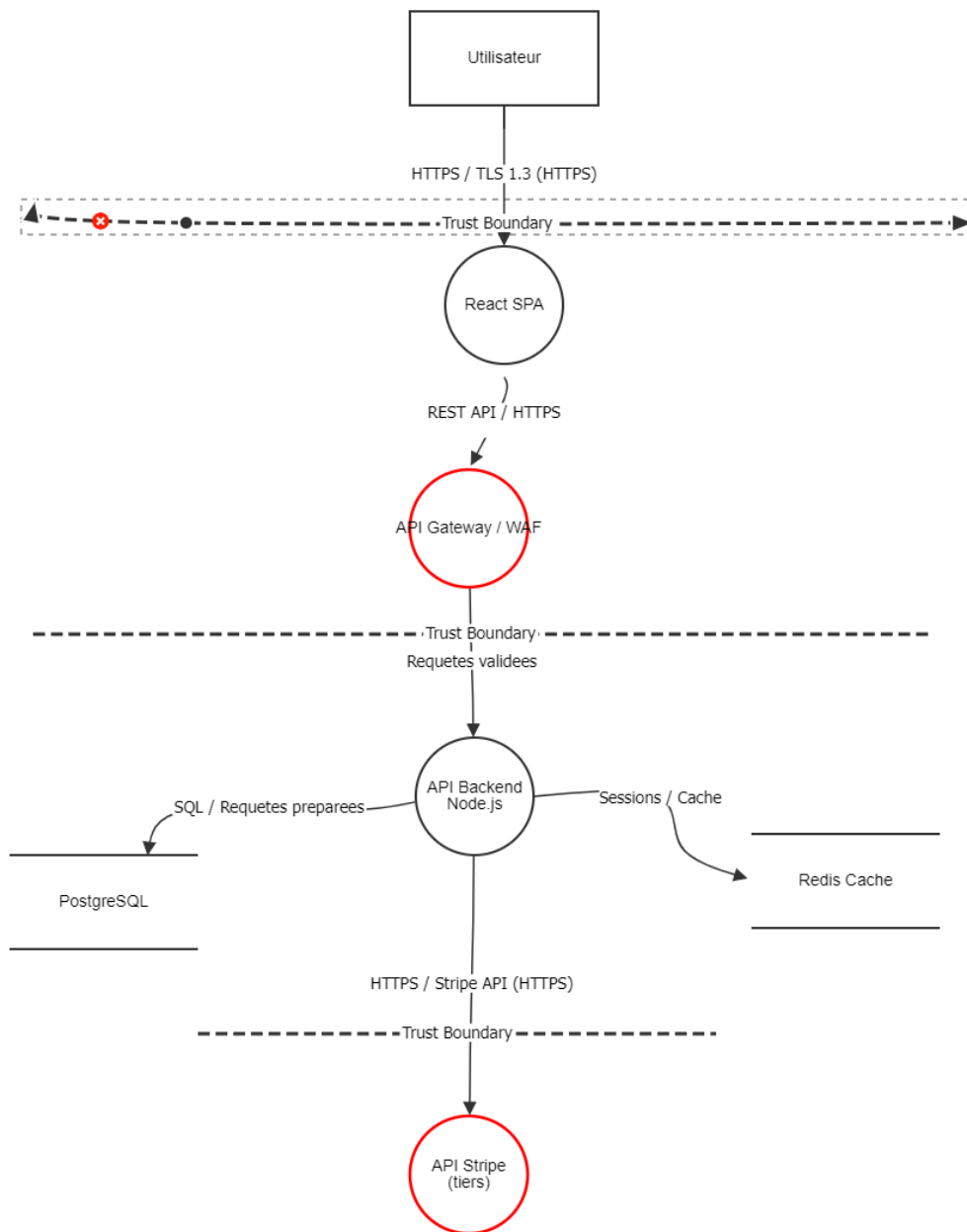
L'application est organisée en quatre modules de routes (auth, products, orders, users) qui interrogent la base SQLite. Le client envoie ses identifiants, reçoit un token JWT, puis l'utilise pour les routes protégées via un middleware d'authentification. En développement, la configuration passe par un fichier .env non commité. En CI, GitHub Actions scanne le code à chaque push avant de construire l'image Docker.



2. Threat Modeling

Le DFD a été modélisé dans OWASP Threat Dragon (fichier threat-model_koumba-awakane.json). Il couvre l'architecture cible de E-Shop Pro, avec des composants prévus pour une mise en production complète (API Gateway/WAF, cache Redis, base PostgreSQL, paiement Stripe). L'implémentation actuelle du dépôt reste plus simple (SQLite, sans gateway ni cache), mais les menaces identifiées restent valables et recourent directement le code audité en Partie 3.

Le diagramme comprend un acteur (Utilisateur), quatre processus (React SPA, API Gateway/WAF, API Backend Node.js, API Stripe) et deux data stores (PostgreSQL, Redis Cache), séparés par trois frontières de confiance : entre l'utilisateur et le frontend, entre la couche publique et le backend interne, et entre le backend et Stripe.



L'analyse STRIDE a fait ressortir 23 menaces sur 6 catégories. Les plus importantes :

Composant/Flux	Catégorie	Menace	Sévérité
API Gateway/WAF	Elevation of Privilege	JWT accepté sans vérification d'algorithme	Critical

Composant/Flux	Catégorie	Menace	Sévérité
API Backend	Tampering	Prix accepté depuis le client	Critical
API Backend	Information Disclosure	Secrets codés en dur	Critical
PostgreSQL	Elevation of Privilege	IDOR sur les commandes	Critical
PostgreSQL	Tampering	Mots de passe en clair	Critical
Flux Backend→Stores	Tampering	Injection SQL par concaténation	Critical
API Backend	Elevation of Privilege	Mass Assignment via req.body	High
API Backend	Information Disclosure	CORS ouvert (origin: '*')	High
React SPA	Spoofing	XSS stocké	High
API Gateway/WAF	Denial of Service	Absence de rate limiting	High

La liste complète des 23 menaces, sur 6 catégories STRIDE, est dans le fichier Threat Dragon joint.

Les trois menaces jugées les plus critiques pour ce projet, compte tenu du contexte e-commerce :

1. **La manipulation du prix côté client.** L'application traite de vrais montants. Si le prix vient du client sans vérification, la fraude est immédiate.
2. **L'injection SQL.** Une base compromise expose tous les comptes, mots de passe et commandes en une seule faille.
3. **Les secrets codés en dur.** Le dépôt doit être public pour cet examen, donc un secret oublié en dur devient exploitable dès la publication.

3. Analyse OWASP Top 10

L'analyse manuelle du code, avant tout outil, a permis d'identifier ces vulnérabilités principales :

Fichier	Ligne	OWASP 2021	CWE	Description	Sévérité
products.js	11	A03 Injection	CWE-89	Recherche produit concaténée dans la requête SQL	Critical
products.js	24, 27	A03 Injection	CWE-89	Mise à jour produit construite par concaténation	Critical
products.js	21-27	A01 Broken Access Control	CWE-915	req.body appliqué sans whitelist (mass assignment)	High
orders.js	22-28	A01 Broken Access Control	CWE-639	Accès à une commande sans vérifier le propriétaire	Critical
orders.js	32-39	A08 Software/Data Integrity	CWE-807	Prix reçu du client, jamais révérifié	Critical
orders.js	48-59	A03 Injection	CWE-79	Commentaire utilisateur renvoyé sans encodage	High
orders.js / users.js	13 / 11	A02 Cryptographic Failures	CWE-798	Secret JWT codé en dur, différent de celui de signature	Critical
users.js	19-26	A01 Broken Access Control	CWE-862	Liste des utilisateurs sans vérifier le rôle admin	Critical

Fichier	Ligne	OWASP 2021	CWE	Description	Sévérité
users.js	29-35	A01 Broken Access Control	CWE-639	Accès au profil d'un autre utilisateur	High
database.js	18-26	A02 Cryptographic Failures	CWE-256	Mots de passe stockés en clair	Critical
server.js	24-28	A05 Security Misconfiguration	CWE-942	CORS ouvert à toutes les origines	High
server.js	37-43	A05 Security Misconfiguration	CWE-209	Stack trace renvoyée en erreur	High
auth.js	36-47	A07 Authentication Failures	CWE-307	Aucun rate limiting sur login/register	Medium

Côté entrées utilisateur, aucune n'était validée avant correction : ni le paramètre de recherche produit, ni le corps des requêtes de mise à jour, création de commande ou commentaire, ni les identifiants d'accès aux ressources (id dans l'URL). Les conséquences vont de l'injection SQL à l'accès non autorisé aux données d'un autre utilisateur.

Sur la gestion des secrets : server.js et auth.js chargeaient correctement les secrets via process.env et un .env.example sans valeurs réelles, .env étant bien exclu du dépôt. Le problème venait d'ailleurs : orders.js et users.js vérifiaient les tokens avec une chaîne codée en dur, différente du secret de signature. Deux comptes de test avec mot de passe en clair étaient aussi présents dans database.js. Les corrections apportées en Partie 7 unifient tout sur process.env.JWT_SECRET et ajoutent GitLeaks à la pipeline pour éviter que ça se reproduise. Le hachage des mots de passe reste dans le plan de remédiation.

4. Règles Semgrep custom

Cinq règles ont été écrites dans .semgrep/rules.yaml, testées directement contre le code réel :

- **sql-injection-template-literal** : détecte une variable interpolée dans un template SQL passé à db.all/db.get/db.run. Trouvée dans products.js, corrigée par des requêtes paramétrées.

- **xss-stored-unencoded-reflection** : détecte un champ de req.body renvoyé tel quel en JSON. Trouvée dans orders.js sur le commentaire de commande.
- **hardcoded-jwt-secret** (avec metavariable-regex) : détecte une chaîne littérale passée à jwt.verify/jwt.sign. Trouvée dans orders.js et users.js.
- **mass-assignment-req-body** (avec pattern-not pour exclure le cas filtré via _.pick()) : détecte req.body appliqué sans filtrage. Trouvée dans products.js.
- **idor-missing-user-check** : détecte une requête par id sans vérification de propriétaire.

Testées localement (semgrep --config .semgrep/ .), ces règles ont produit 8 détections sans doublon ni faux positif sur le code initial. Après corrections, plus aucune ne se déclenche, à l'exception d'un faux positif rencontré et documenté (voir Partie 6).

5. Pipeline DevSecOps

La pipeline compte 7 jobs. GitLeaks scanne les secrets en premier, puis Semgrep, Snyk et Trivy filesystem tournent en parallèle. Le build Docker ne se lance que si ces quatre jobs passent. Vient ensuite le scan Trivy de l'image, puis un résumé final. Chaque scan uploades son rapport SARIF vers Security → Code scanning, qu'il bloque ou non.

The screenshot displays a GitHub Actions workflow run for the repository 'AwaKane / eshop-supdeco'. The workflow is titled 'correction du dockerfile #9' and has a status of 'Succès' (Success). The total duration is '2 min 43 s' and there are '2' artifacts. The workflow steps are listed in a summary bar:

- Scannez les secrets — GitLe... 12s
- SAST — Semgrep 32 secondes
- SCA — Dépendances de Snyk 57s
- SCA — Système de fic... années 30
- Création d'une image Docker 26s
- Trivy — Numérisation ... 1 min 5 s
- Résumé de sécurité 4s

The left sidebar shows the 'Summary' tab with a list of jobs: Scan secrets — GitLeaks, SAST — Semgrep, SCA — Snyk Dependencies, SCA — Trivy Filesystem, Build — Docker Image, Trivy — Image Scan, and Security Summary. The main content area shows the 'devsecops.yml' file with the instruction 'sur: appuyer'.

Réponses aux Questions

Q5.1 Sur quel(s) job(s) avez-vous mis exit-code: '1' et pourquoi ?

exit-code: '1' est posé sur le scan Trivy de l'image Docker finale, restreint aux vulnérabilités CRITICAL corrigéables (ignore-unfixed: true). C'est la dernière étape avant un déploiement potentiel : pour une application e-commerce traitant des données de paiement, livrer une image avec une vulnérabilité critique connue et évitable est le risque le moins justifiable, d'où le blocage strict à ce stade, alors que les scans plus en amont restent informatifs pour ne pas ralentir chaque commit sur du bruit non critique.

Q5.2 Un développeur pousse accidentellement une clé API hardcodée. Que se passe-t-il, étape par étape ?

1. Le push déclenche la pipeline.
2. Le job gitleaks s'exécute en premier avec l'historique Git complet et détecte la clé, retournant un code de sortie non nul.
3. Le rapport SARIF est uploadé vers Security → Code scanning même si le job échoue (if: always()).
4. Le job gitleaks passe en Failed.
5. Le job build, qui dépend de gitleaks via needs:, ne se déclenche jamais.
6. Aucune image Docker n'est construite avec la clé compromise.
7. Le développeur voit l'échec et l'alerte détaillée, doit retirer la clé, la révoquer côté fournisseur, et repousser un correctif.

Q5.3 Différence entre CVE et CWE, avec un exemple de chaque ?

Un **CWE** décrit une catégorie générique de faiblesse logicielle, indépendante de tout logiciel précis. exemple : **CWE-89 (SQL Injection)**, détecté par Semgrep dans products.js. Une **CVE** identifie une vulnérabilité précise dans un produit/version donné

exemple : les CVE remontées par Snyk sur les dépendances (ex. la faille de confusion d'algorithme corrigée par la montée de version de jsonwebtoken vers 9.0.3) et par Trivy sur l'image (CVE-2026-31789 sur OpenSSL, CVE-2026-59873 sur tar). En résumé : le CWE classe le type de faille, la CVE identifie une instance précise et datée de cette faille.

6. Résultats

Après le premier passage de la pipeline, les alertes principales étaient :

Outil	Où	Vulnérabilité	Sévérité	Statut
Semgrep	Dockerfile	Conteneur exécuté en root	High	Fixed

Outil	Où	Vulnérabilité	Sévérité	Statut
Semgrep	orders.js, users.js	Secret JWT codé en dur	Critical	Fixed
Semgrep	products.js	Injection SQL (recherche et mise à jour)	Critical	Fixed
GitLeaks	—	Aucun secret dans l'historique	—	Passed
Snyk	package.json	46 issues sur axios, lodash, multer, sqlite3, express	High/Critical	Fixed
Trivy image	Base node:18-alpine3.20	OpenSSL, CVE-2026-31789	Critical	Fixed
Trivy image	tar intégré à l'image	CVE-2026-59873	Critical	Fixed

Security and quality

- Overview
- Findings
 - Dependabot
 - Malware
 - Vulnerabilities
 - Code scanning** 33
 - Secret scanning
- Reporting
 - Security policy
 - Advisories

Code scanning

🟢 All tools are working as expected Tools 5 + Add tool

🔍 is:open branch:main

33 Open 4,924 Closed Language Tool Rule Severity Sort

- 🔒 **tar: node-tar: Denial of Service via crafted long-path tar archive** (High) Library main
#4952 opened il y a 3 heures • Detected by Trivy in usr/.../tar/package.json:1
- 🔒 **ip-address: ip-address: Inconsistent IP address parsing leads to Server-Side Request Forgery (SSRF) and trust-boundary bypass** (High) Library main
#4791 opened le mois dernier • Detected by Trivy in usr/.../ip-address/package.json:1
- 🔒 **brace-expansion: DoS via unbounded intermediate arrays, bypassing the CVE-2026-14257 mitigation** (High) Library main
#4780 opened le mois dernier • Detected by Trivy in usr/.../brace-expansion/package.json:1
- 🔒 **brace-expansion: Brace-expansion: Denial of Service via memory exhaustion in expand()** (High) ...

Sur les dépendances : axios, multer et bcrypt n'étaient utilisés nulle part dans le code, vérifié par recherche exhaustive. Les supprimer a réglé la majorité des 46 issues Snyk sans aucun risque. sqlite3, express et jsonwebtoken ont été mis à jour vers des versions corrigées. Les deux CVE critiques sur l'image Docker ne venaient ni du code ni des dépendances npm, mais du vieillissement de l'image de base elle-même (Node 18 et Alpine 3.20, tous deux en fin de vie), corrigées en passant sur node:24-alpine.

L'analyse manuelle avait trouvé ce que les outils ne voient pas facilement : l'IDOR, la manipulation du prix, les mots de passe en clair, le CORS trop ouvert. À l'inverse, les outils ont trouvé le Dockerfile root et les 46 vulnérabilités de dépendances, invisibles à l'œil nu. Fait notable : les règles génériques p/owasp-top-ten et p/nodejs (170 règles) n'ont remonté qu'une seule alerte sur tout le projet, contre 5 pour les règles écrites spécifiquement pour ce code.

Un faux positif a été rencontré après correction : la règle sql-injection-template-literal continuait à se déclencher sur la requête de mise à jour produit, alors que le nom de colonne interpolé venait exclusivement d'une whitelist figée dans le code, jamais d'une entrée utilisateur. La règle, purement syntaxique, ne pouvait pas voir cette nuance. Plutôt que d'assouplir la règle et risquer de rouvrir un vrai trou, le cas a été documenté directement dans le code avec un commentaire `// nosemgrep` et une justification écrite.

7. Corrections et validation

Secret JWT codé en dur (orders.js:13, users.js:11). Le code vérifiait les tokens avec une chaîne en dur au lieu de `process.env.JWT_SECRET`, incohérente avec le secret de signature utilisé dans `auth.js`. Corrigé en utilisant partout la variable d'environnement, avec l'algorithme explicitement forcé (`{ algorithms: ['HS256'] }`)), ce qui règle en même temps la menace d'algorithme falsifié identifiée en Partie 2.

☐ ☒ **Semgrep Finding: semgrep.hardcoded-jwt-secret** Error

#4916 closed as fixed 9 hours ago • Detected by Semgrep OSS in `src/routes/users.js:11`

☐ ☒ **Semgrep Finding: semgrep.hardcoded-jwt-secret** Error

#4911 closed as fixed 9 hours ago • Detected by Semgrep OSS in `src/routes/orders.js:13`

Injection SQL sur la recherche produit (products.js:7-17). La requête concaténait directement le paramètre de recherche. Corrigée avec des placeholders `?`, qui empêchent toute valeur injectée d'être interprétée comme du SQL.

Mass assignment et injection SQL sur la mise à jour produit (products.js:20-33). `req.body` était appliqué en entier, permettant de modifier n'importe quel champ, et la requête était construite par concaténation. Corrigé avec `_.pick()` pour ne garder que les champs autorisés (nom, description, prix, stock), combiné à des requêtes paramétrées.

← Alertes de scan de code

Résultat Semgrep : semgrep.mass-assignment-req-body # 4914



En succursale principal • il y a 9 heures

src/routes/ products.js :21

```
18
19 // ✖ VULN 2 – Mass Assignment : req.body appliqué directement
20 router.put (('/:id' , ( req , res ) => {
21   const mises à jour = req.corps; // ✖ L'attaquant peut passer isAdmin:true, price:0, etc.
```

⚠ Avertissement

Semgrep Finding: semgrep.mass-assignment-req-body

Potentiel d'affectation de masse : req.body appliqué sans whitelist des champs autorisés.

Semgrep OSS

```
22
23   const champs = Object.keys ( updates )
24   .map ( k => ` ${ k } = ' ${ updates [ k ] } '` ) // ✖ Injection SQL possible
```

IDOR sur les commandes et les profils (orders.js:20-29, users.js:28-35). N'importe quel utilisateur authentifié pouvait consulter la commande ou le profil de n'importe qui d'autre. Corrigé en vérifiant que l'id de la ressource correspond bien à l'utilisateur connecté avant de la renvoyer.

[← Code scanning alerts](#)

Semgrep Finding: semgrep.idor-missing-user-check #4900

 Fixed

On branch `main` • 9 hours ago

src/routes/orders.js:24 

```
21 // Un utilisateur peut accéder aux commandes de N'IMPORTE QUEL autre utilisateur
22 router.get('/:id', auth, (req, res) => {
23   // ✗ Pas de vérification que req.params.id === req.user.id
24   db.get('SELECT * FROM orders WHERE id = ?', [req.params.id], (err, order) => {
25     if (err) return res.status(500).json({ error: err.message });
26     if (!order) return res.status(404).json({ error: 'Commande introuvable' });
27     res.json(order); // Retourne la commande de n'importe quel utilisateur
28   });
```

 Warning

Semgrep Finding: semgrep.idor-missing-user-check

IDOR potentiel – vérifier que la ressource appartient à req.user.id avant de la retourner.

Semgrep OSS

```
29   });
30
31   // ✗ VULN 2 – Tampering : l'API utilise le prix envoyé par le client
```

Bonus, découvert en cours de correction : la création de commande cumulait manipulation de prix et injection SQL (orders.js:31-45). Le prix n'est plus jamais lu depuis le client, il est relu en base à partir du produit, et la requête est paramétrée.

Toutes ces corrections ont été vérifiées avec Semgrep, qui ne remonte plus aucune de ces alertes.

Security and quality

Overview

Findings

Dependabot

Malware

Vulnerabilities

Code scanning 33

Secret scanning

Reporting

Security policy

Advisories

Code scanning

✓ All tools are working as expected

Tools 5 + Add tool

Q is:closed branch:main tool:"Snyk Open Source"

✕ Clear current search query, filters, and sorts

0 Open

46 Closed

Closed as

Language

Tool

Rule

Severity

Sort

✓ Critical severity - Uncaught Exception vulnerability in multer Critical

#22 closed as fixed 4 hours ago • Detected by Snyk Open Source in package.json:1

main

✓ Critical severity - Predictable Value Range from Previous Values vulnerability in form-data Critical

#14 closed as fixed 4 hours ago • Detected by Snyk Open Source in package.json:1

main

✓ Critical severity - Prototype Pollution vulnerability in axios Critical

#5 closed as fixed 4 hours ago • Detected by Snyk Open Source in package.json:1

main

✓ Critical severity - HTTP Response Splitting vulnerability in axios Critical

#1 closed as fixed 4 hours ago • Detected by Snyk Open Source in package.json:1

main

Pour ce qui reste à faire : les mots de passe en clair, le CORS ouvert, la stack trace exposée en erreur, le XSS stocké et l'absence de rate limiting sont documentés dans un plan de remédiation, priorisés par sévérité et complexité, avec des délais indicatifs allant de 3 jours à deux semaines.

Le Dockerfile a aussi été revu en profondeur : image de base passée à node:24-alpine (Node 24 étant la version LTS actuelle, contre Node 18 en fin de vie), ajout d'un utilisateur non-root, copie limitée au strict nécessaire via un .dockerignore, npm ci à la place de npm install, suppression du port de debug 9229 et du mode --inspect. Un compromis à noter : le tag node:24-alpine n'est pas figé sur un patch exact, ce qui garde l'image à jour sur les correctifs système à chaque rebuild, au prix d'une reproductibilité un peu moindre. C'est un choix direct en réaction aux deux CVE critiques découvertes sur l'image de base initialement figée.

8. Bilan

En auditant mon propre projet, j'ai réalisé que la sécurité en théorie et la sécurité dans le code réel, ce n'est pas pareil. Je pensais connaître les failles de cette application depuis le TP3, mais l'analyse manuelle et les outils m'ont fait découvrir des choses que je n'avais pas vues, comme l'incohérence entre le secret utilisé pour signer les tokens et celui utilisé pour les vérifier, ou une injection SQL supplémentaire trouvée par hasard en corrigeant autre chose.

Ce qui m'a le plus marquée, c'est l'écart entre les règles génériques de Semgrep et mes propres règles écrites après avoir vraiment lu le code : les règles génériques n'ont trouvé qu'une seule alerte sur tout le projet, contre cinq pour les miennes. Ça m'a fait comprendre pourquoi l'énoncé insiste sur des règles adaptées à son propre code plutôt que de compter sur des scanners standards.

J'ai aussi eu un faux positif généré par ma propre règle, ce qui m'a appris qu'écrire une règle ne s'arrête pas au moment où elle détecte quelque chose. Il faut la tester contre le code réel, comprendre pourquoi

elle se déclenche, et documenter les cas où elle a tort au lieu de l'assouplir sans réfléchir, au risque de rouvrir une vraie faille.

Enfin, la pipeline m'a appris une leçon que je n'attendais pas : Trivy a trouvé des vulnérabilités critiques dans l'image Docker qui ne venaient ni de mon code ni de mes dépendances, mais de l'image de base elle-même, devenue obsolète avec le temps. Je n'avais jamais réalisé qu'une image figée pouvait devenir vulnérable sans qu'on touche à une seule ligne de code.

Si je recommençais un projet depuis le début, je ferais trois choses différemment : écrire mes règles Semgrep en même temps que le code plutôt qu'après coup, prévoir un scan Trivy régulier plutôt que seulement à chaque push, et hasher les mots de passe dès le premier commit plutôt que de le laisser traîner dans un plan de remédiation. Une mauvaise pratique déjà en place est toujours plus dure à corriger que si elle n'avait jamais existé.